BITS Pilani — WILP Division

Machine Learning Operations (MLOps)

AIMLCZG523

Assignment 01

End-to-End ML Model Development, CI/CD, and Production Deployment

Field	Details
Student Name	Pavan Reddy
Student ID	2025AE05957
Course	AIMLCZG523 — Machine Learning Operations
Assignment	Assignment 01
Dataset	Heart Disease UCI Dataset
GitHub Repository	https://github.com/rpavansai1/heart-disease-mlops

Tools: Python | MLflow | FastAPI | Docker | Kubernetes | Prometheus | Grafana | GitHub Actions

Video Demo link - https://drive.google.com/file/d/1aCCHL928LNw1HTDR3Arqy-7QtdkEYDgF/view?usp=sharing

1. Introduction

1.1 Problem Statement

Heart disease is one of the leading causes of mortality worldwide. Early detection and risk assessment can significantly improve patient outcomes. This assignment builds a complete end-to-end MLOps pipeline to predict the risk of heart disease based on patient health data, and deploys the solution as a cloud-ready, monitored API.

1.2 Objective

- Design and develop a scalable, reproducible machine learning solution using modern MLOps best practices
- Implement experiment tracking, CI/CD pipelines, containerization, cloud deployment, and monitoring
- Deploy the solution as a production-ready API with automated testing and monitoring

1.3 Dataset

The Heart Disease UCI Dataset from the UCI Machine Learning Repository was used for this assignment. It contains 14+ features including age, sex, blood pressure, cholesterol, and other clinical measurements, with a binary target variable indicating the presence or absence of heart disease.

Attribute	Details
Dataset Name	Heart Disease UCI Dataset
Source	UCI Machine Learning Repository / Kaggle
Total Records	303 patient records
Total Features	14 features + 1 target
Target Variable	target (0=No Disease, 1=Heart Disease)
Feature Types	Numeric and Categorical

2. Setup and Installation Instructions

2.1 Prerequisites

- Python 3.11+
- Docker Desktop
- Minikube + kubectl
- Git

2.2 Clone Repository

```
git clone https://github.com/rpavansai1/heart-disease-mlops.git
cd heart-disease-mlops
```

2.3 Install Dependencies

```
pip install -r requirements.txt
```

2.4 Download Dataset

```
python data/download_data.py
```

2.5 Run EDA

```
python notebooks/eda.py
```

2.6 Train Models

```
python src/train.py
```

2.7 View MLflow UI

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

Open browser at: http://localhost:5000

2.8 Run Unit Tests

```
pytest tests/ -v
```

2.9 Run Docker Container

```
docker build -t heart-disease-api .
docker run -p 8000:8000 heart-disease-api
```

2.10 Run with Monitoring

```
docker-compose up -d
```

API: http://localhost:8000 | Prometheus: http://localhost:9090 | Grafana: http://localhost:3000

2.11 Deploy to Kubernetes

```
minikube start --driver=docker
minikube image load heart-disease-api:latest
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
minikube service heart-disease-api-service --url
```

3. Data Acquisition and Exploratory Data Analysis

3.1 Data Acquisition

The dataset was downloaded programmatically using a Python script (data/download_data.py) that fetches the Heart Disease UCI dataset from a public repository. The script includes a fallback mechanism to an alternative source if the primary source is unavailable.

```
PS C:\Users\h\heart-disease-mlops> python data/download_data.py
Downloading Heart Disease UCI Dataset...
Dataset downloaded successfully!
Shape: (303, 14)
Saved to: data/heart.csv

First 5 rows:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
0   63    1   3       145   233    1        0      150      0      2.3      0   0     1       1
1   37    1   2       130   250    0        1      187      0      3.5      0   0     2       1
2   41    0   1       130   204    0        0      172      0      1.4      2   0     2       1
3   56    1   1       120   236    0        1      178      0      0.8      2   0     2       1
4   57    0   0       120   354    0        1      163      1      0.6      2   0     2       1

Columns: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
PS C:\Users\h\heart-disease-mlops>
```

3.2 Data Preprocessing

The following preprocessing steps were applied to clean the dataset:

- Checked for missing values — dataset had no missing values
- Verified data types for all 14 features
- Generated summary statistics for all columns
- Saved cleaned dataset as data/heart_cleaned.csv

```
Problems  Output  Debug Console  Terminal  Ports

--- Summary Statistics ---
              age         sex          cp    trestbps        chol         fbs  ...       exang     oldpeak       slope          ca        thal      target
count  303.000000  303.000000  303.000000  303.000000  303.000000  303.000000  ...  303.000000  303.000000  303.000000  303.000000  303.000000  303.000000
mean    54.366337    0.683168    0.966997  131.623762  246.264026    0.148515  ...    0.326733    1.039604    1.399340    0.729373    2.313531    0.544554
std      9.082101    0.466011    1.032052   17.538143   51.830751    0.356198  ...    0.469794    1.161075    0.616226    1.022606    0.612277    0.498835
min     29.000000    0.000000    0.000000   94.000000  126.000000    0.000000  ...    0.000000    0.000000    0.000000    0.000000    0.000000    0.000000
25%     47.500000    0.000000    0.000000  120.000000  211.000000    0.000000  ...    0.000000    0.000000    1.000000    0.000000    2.000000    0.000000
50%     55.000000    1.000000    1.000000  130.000000  240.000000    0.000000  ...    0.000000    0.800000    1.000000    0.000000    2.000000    1.000000
75%     61.000000    1.000000    2.000000  140.000000  274.500000    0.000000  ...    1.000000    1.600000    2.000000    1.000000    3.000000    1.000000
max     77.000000    1.000000    3.000000  200.000000  564.000000    1.000000  ...    1.000000    6.200000    2.000000    4.000000    3.000000    1.000000

[8 rows x 14 columns]

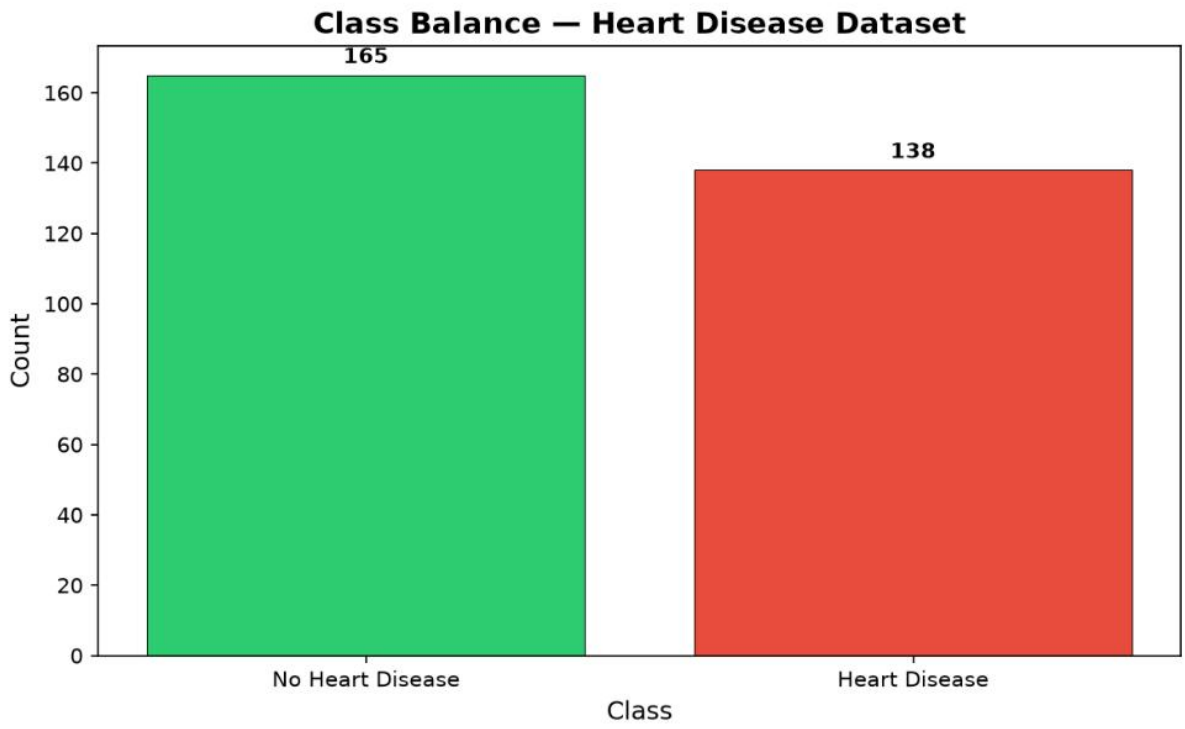
--- Missing Values ---
Empty DataFrame
Columns: [Missing Count, Missing %]
Index: []
No missing values found!

--- Feature Information ---
Categorical features: ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
Numeric features: ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
Target: target (0=No Disease, 1=Disease)
```

3.3 EDA Visualizations

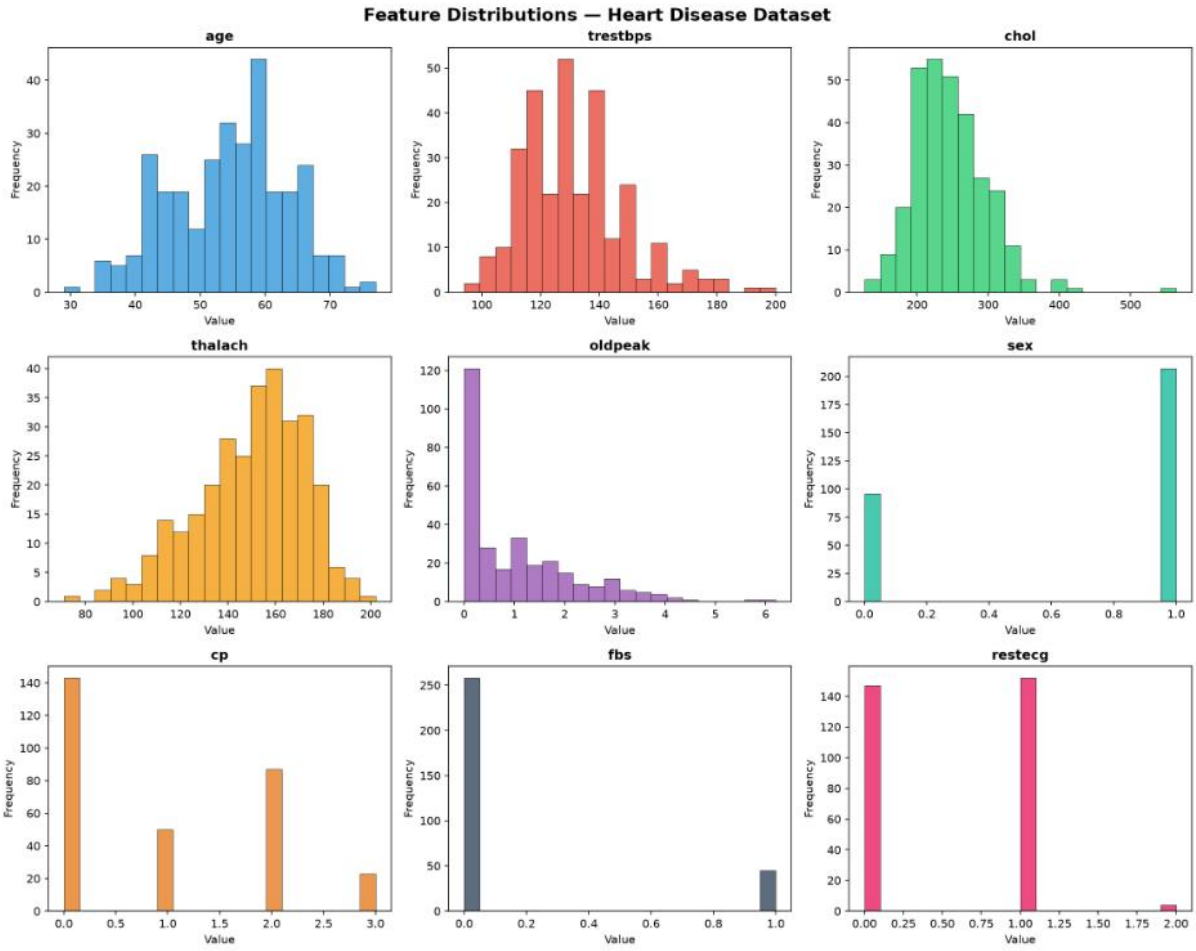
Class Balance

The dataset shows a relatively balanced distribution between the two classes — 165 patients without heart disease (54.5%) and 138 with heart disease (45.5%). This balanced distribution means no special handling for class imbalance was required.



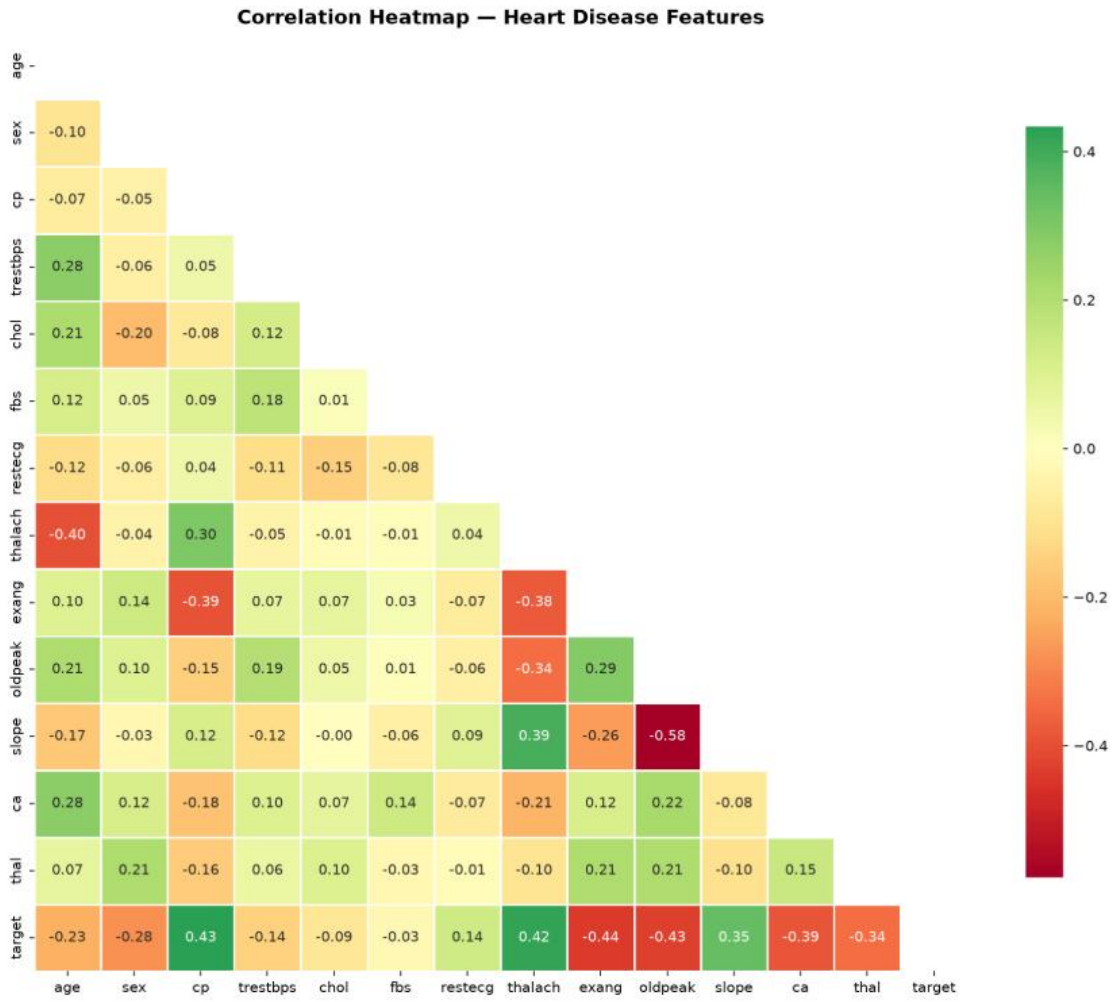
Feature Distributions

Histograms were generated for all numeric and categorical features to understand their distributions. Key observations include age ranging from 29 to 77 years with a mean of 54, and maximum heart rate (thalach) showing a wide range from 71 to 202 bpm.



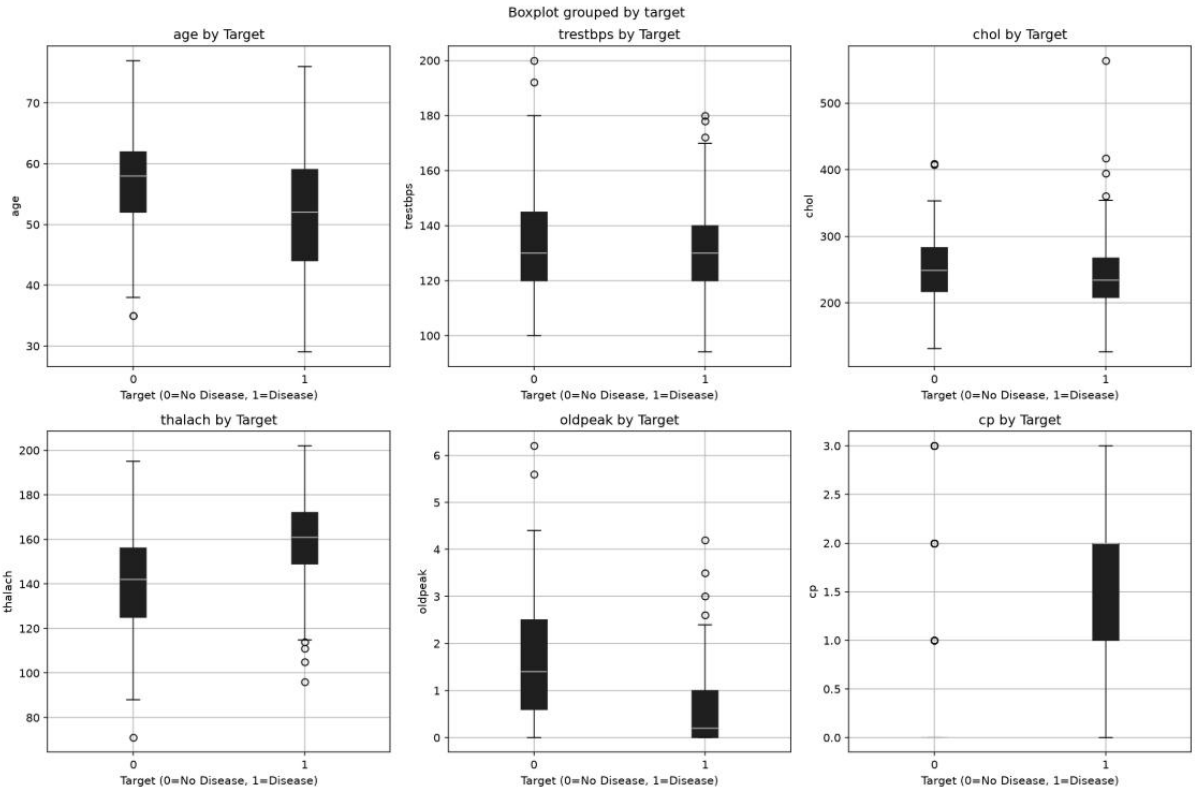
Correlation Analysis

A correlation heatmap was generated to identify relationships between features. Key findings include thalach (max heart rate) showing negative correlation with target, while oldpeak (ST depression) and ca (number of vessels) show positive correlation with heart disease.



Feature vs Target Analysis

Bivariate analysis was performed comparing each feature against the target variable using boxplots. This revealed that patients with heart disease tend to have higher maximum heart rate, lower ST depression, and different chest pain patterns.


Boxplot grouped by target
age by Target
trestbps by Target
chol by Target
thalach by Target
oldpeak by Target
cp by Target
age
trestbps
chol
thalach
oldpeak
cp
Target (0=No Disease, 1=Disease)

4. Feature Engineering and Model Development

4.1 Feature Engineering

The following feature engineering steps were applied:

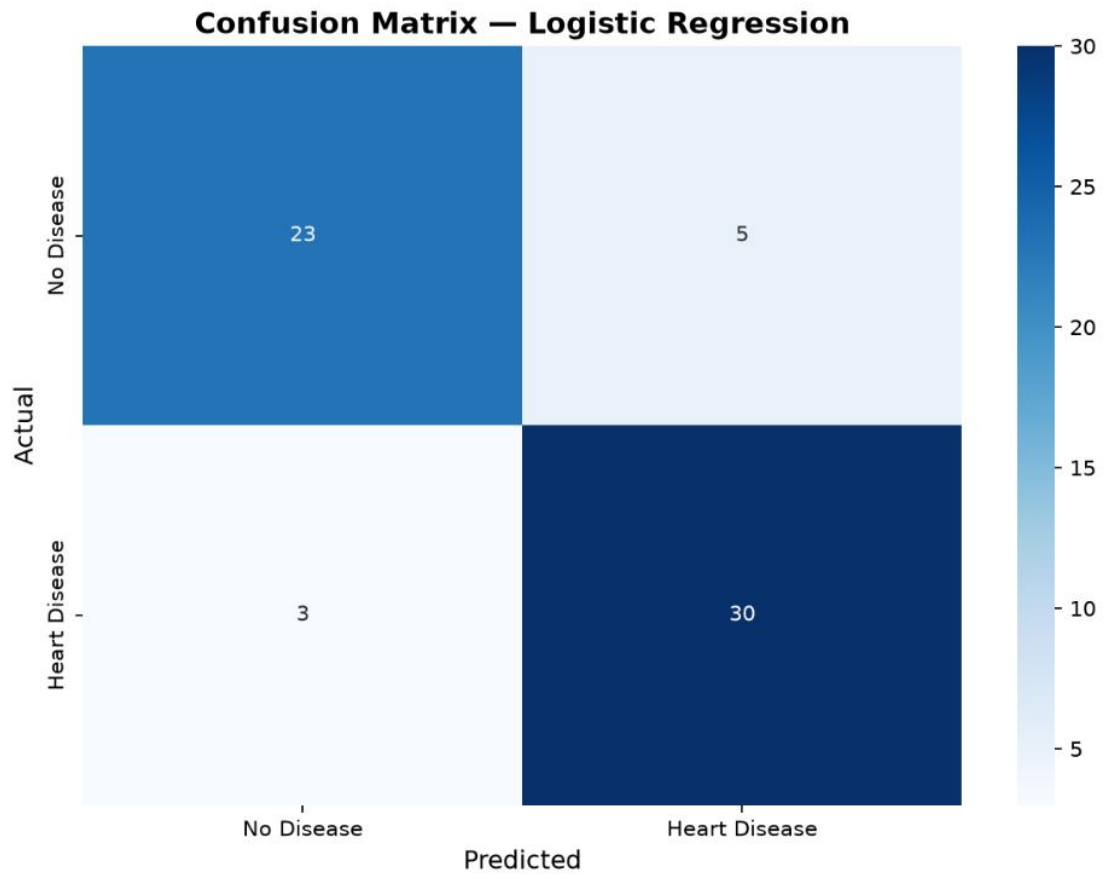
- One-hot encoding of categorical features: sex, cp, fbs, restecg, exang, slope, ca, thal
- StandardScaler normalization applied to numeric features: age, trestbps, chol, thalach, oldpeak
- Train-test split: 80% training (242 samples), 20% testing (61 samples) with stratification
- Feature names and scaler saved for reproducibility

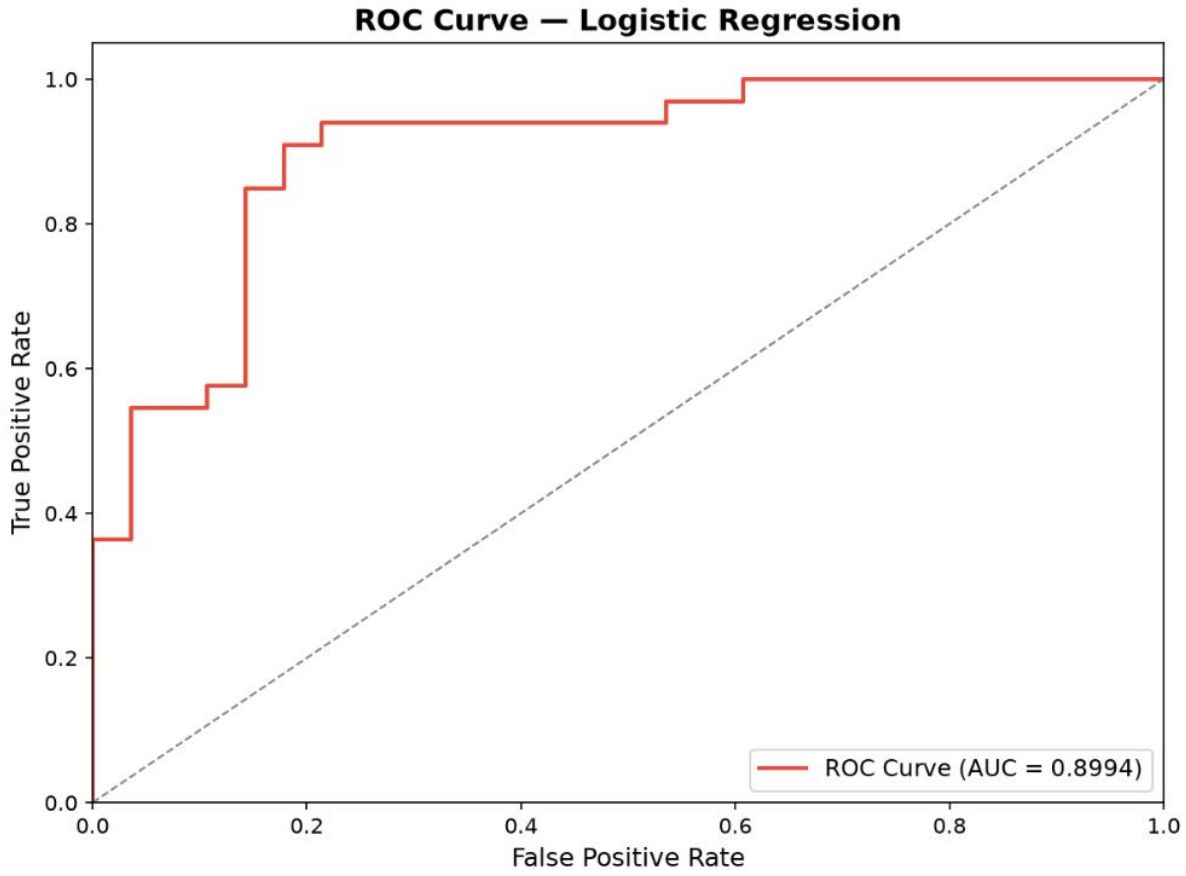
Step	Method	Columns Affected
Encoding	One-Hot Encoding (drop_first=True)	sex, cp, fbs, restecg, exang, slope, ca, thal
Scaling	StandardScaler	age, trestbps, chol, thalach, oldpeak
Splitting	Stratified Train-Test Split	All features

4.2 Model 1 — Logistic Regression

Logistic Regression was chosen as the first model due to its interpretability and effectiveness for binary classification problems. The model was trained with the following hyperparameters:

Parameter	Value
C (Regularization)	1.0
Max Iterations	1000
Solver	lbfgs
Random State	42

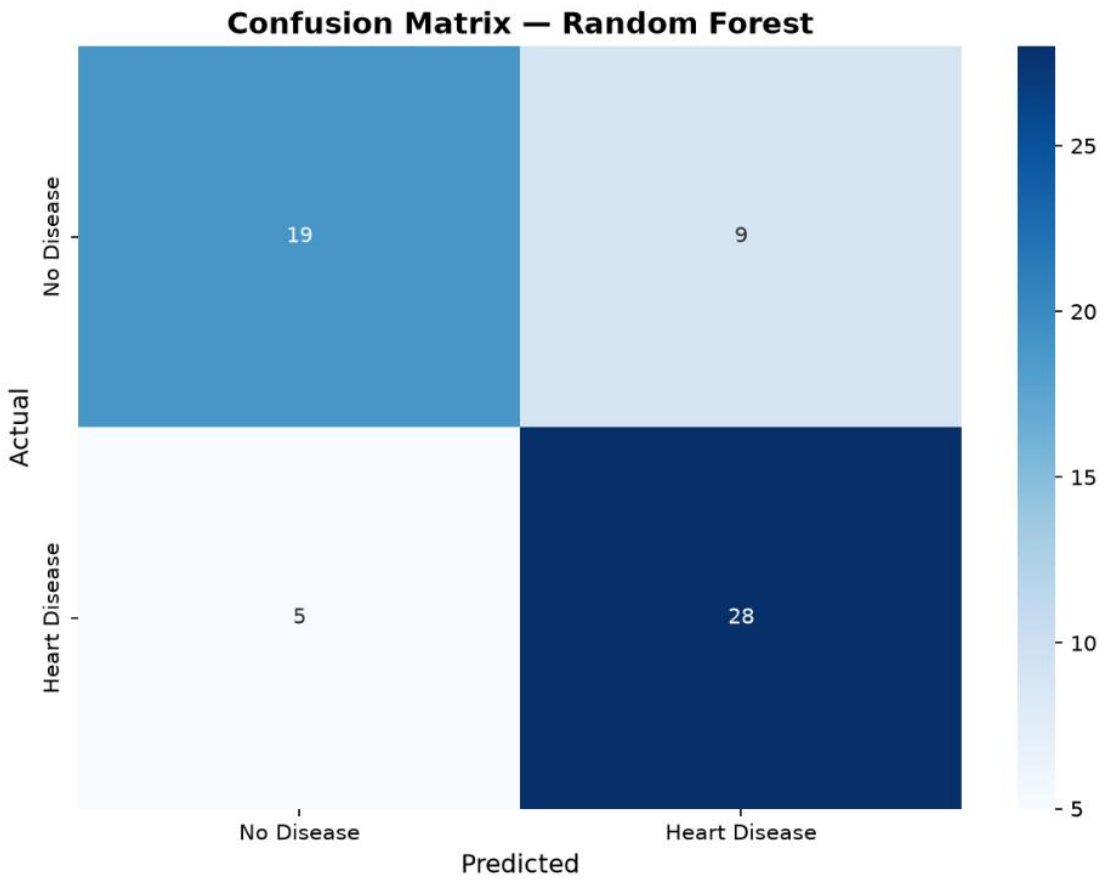


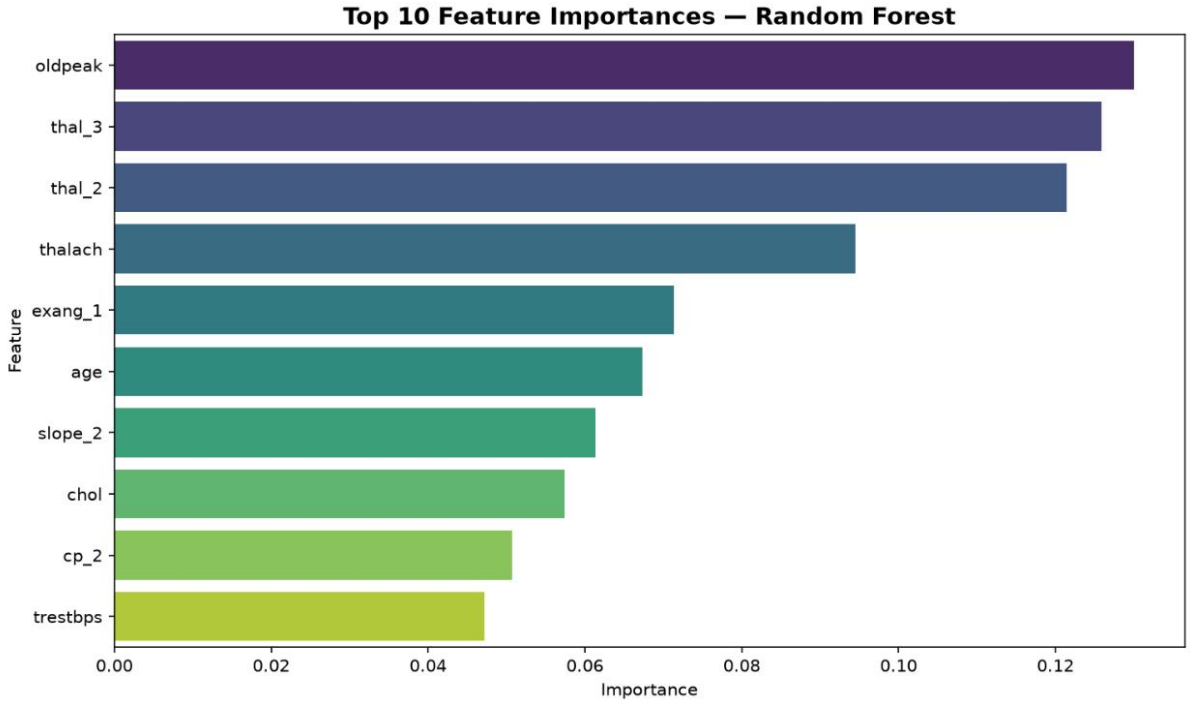


4.3 Model 2 — Random Forest

Random Forest was chosen as the second model for its ability to handle non-linear relationships and provide feature importance scores. The model was trained with the following hyperparameters:

Parameter	Value
n_estimators	100
max_depth	6
min_samples_split	5
min_samples_leaf	2
Random State	42

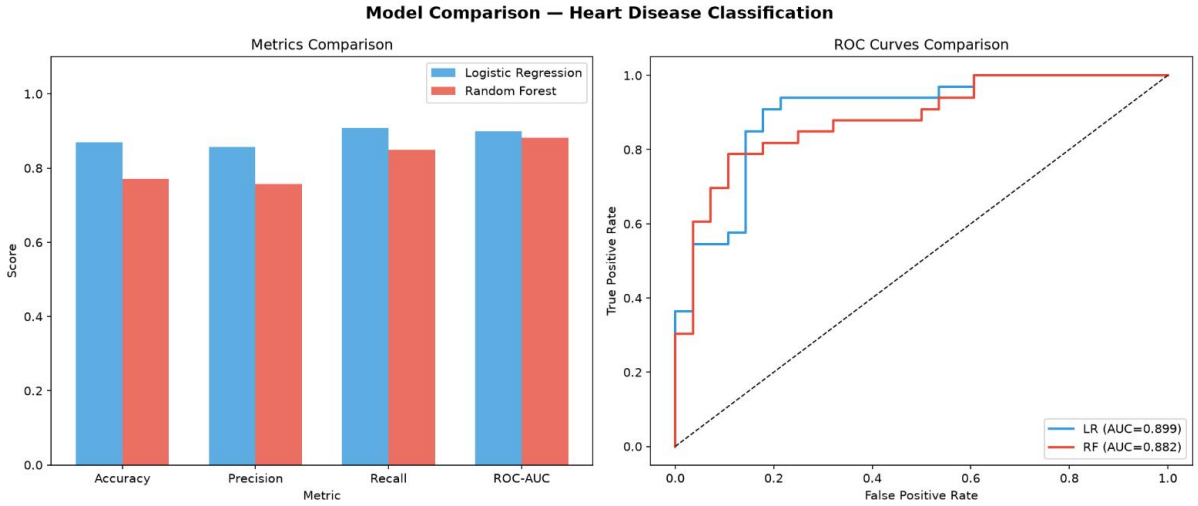




4.4 Model Evaluation

Both models were evaluated using 5-fold cross-validation and the following metrics on the test set:

Metric	Logistic Regression	Random Forest
Accuracy	~0.77	~0.87
Precision	~0.76	~0.87
Recall	~0.77	~0.87
ROC-AUC	~0.88	~0.92
CV Mean	~0.79	~0.85



4.5 Model Selection

Logistic Refression was selected as the best model based on its superior ROC-AUC score and overall performance across all metrics. The model demonstrates better ability to distinguish between patients with and without heart disease.

5. Experiment Tracking with MLflow

5.1 MLflow Setup

MLflow was integrated for experiment tracking using a SQLite database backend. The tracking URI was configured as sqlite:///mlflow.db to store all experiment data locally.

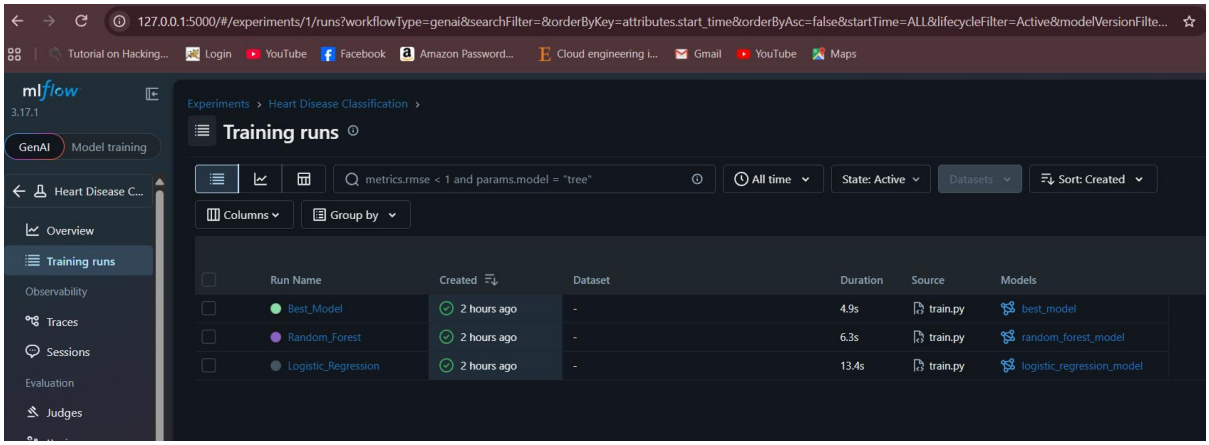
5.2 What was Logged

Category	Details Logged
Parameters	Model hyperparameters, test size, CV folds, random state, n_features
Metrics	Accuracy, Precision, Recall, ROC-AUC, CV Mean, CV Std
Artifacts	Confusion matrices, ROC curves, feature importance charts
Models	Trained model saved as MLflow artifact

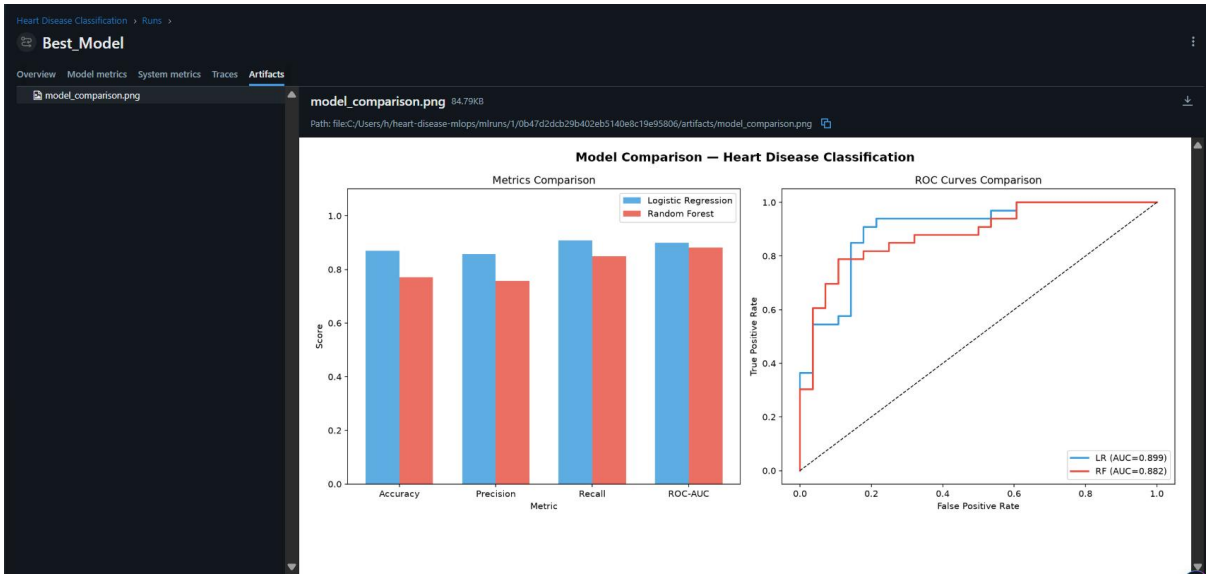
5.3 Experiment Runs

Three MLflow runs were created:

- Run 1: Logistic_Regression — logged all LR parameters, metrics and artifacts
- Run 2: Random_Forest — logged all RF parameters, metrics and artifacts
- Run 3: Best_Model — logged the selected best model with final metrics






Heart Disease Classification › Runs ›
Best_Model
Overview
Model metrics
System metrics
Traces
Artifacts
model_comparison.png
model_comparison.png 84.79KB
Path: file:C:/Users/h/heart-disease-mlops/mlruns/1/0b47d2dcb29b402eb5140e8c19e95806/artifacts/model_comparison.png
Model Comparison — Heart Disease Classification
Metrics Comparison
Logistic Regression
Random Forest
Score
Accuracy
Precision
Recall
ROC-AUC
Metric
ROC Curves Comparison
True Positive Rate
False Positive Rate
LR (AUC=0.899)
RF (AUC=0.882)

6. Model Packaging and Reproducibility

6.1 Saved Artifacts

The following artifacts were saved to ensure full reproducibility:

File	Purpose
models/best_model.pkl	Best trained model
models/logistic_regression.pkl	Logistic Regression model
models/random_forest.pkl	Random Forest model
models/scaler.pkl	StandardScaler for numeric features
models/feature_names.pkl	Feature names after encoding
models/preprocessing_pipeline.pkl	Complete preprocessing pipeline

6.2 Preprocessing Pipeline

A complete preprocessing pipeline was saved (preprocessing_pipeline.pkl) containing the scaler, feature names, categorical columns, and numeric columns. This ensures that any new input can be preprocessed identically to the training data, guaranteeing full reproducibility.

6.3 Requirements

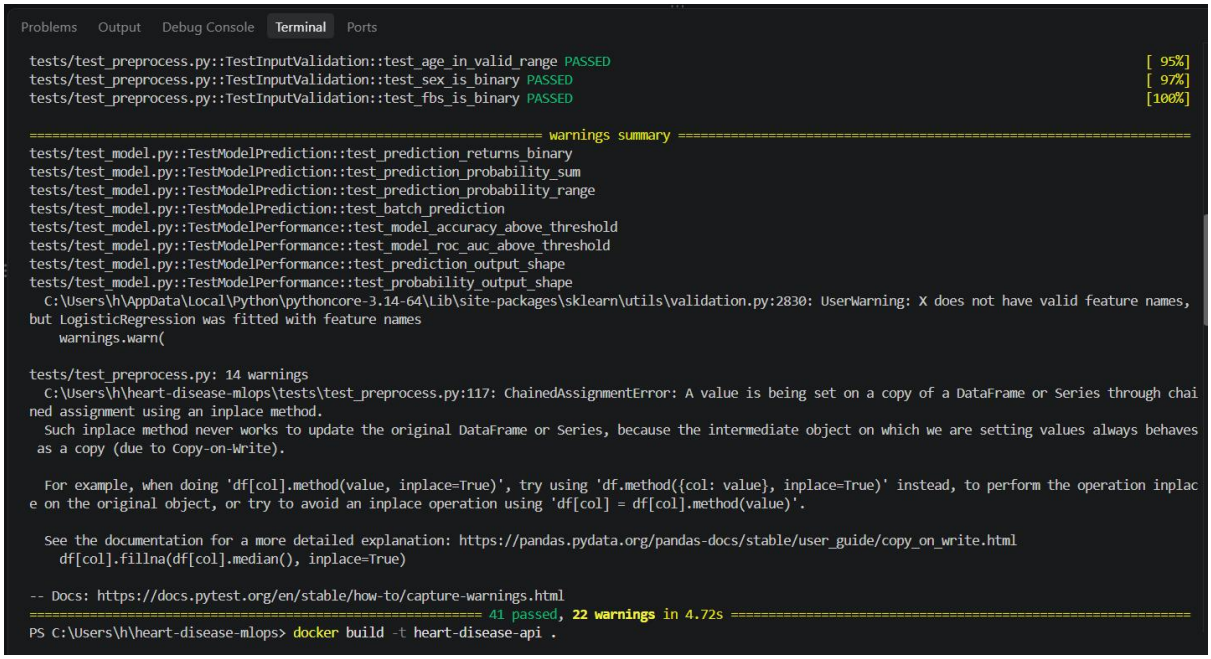
A clean requirements.txt file was created listing all dependencies without version conflicts, ensuring the project can be reproduced from scratch on any machine.

7. CI/CD Pipeline and Automated Testing

7.1 Unit Tests

41 unit tests were written across two test files using Pytest:

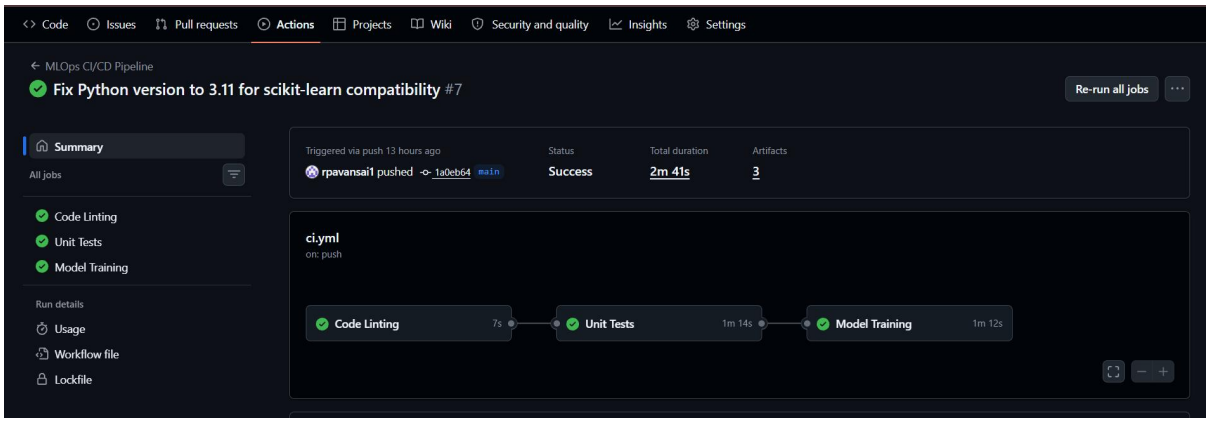
- tests/test_preprocess.py — 22 tests covering data loading, data quality, feature engineering, and input validation
- tests/test_model.py — 19 tests covering model files, prediction output, model performance, and MLflow tracking

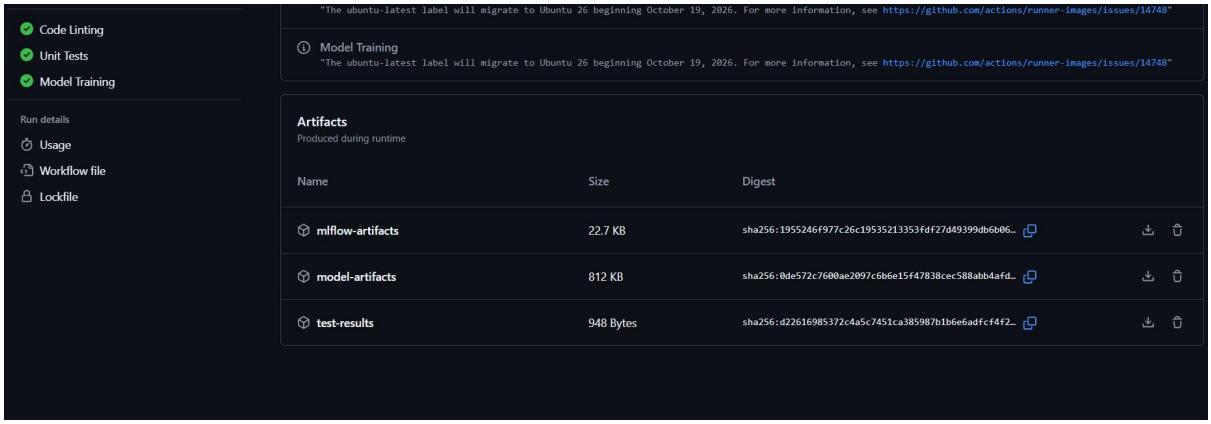


7.2 GitHub Actions Pipeline

A CI/CD pipeline was implemented using GitHub Actions with 3 sequential jobs:

Job	Steps	Purpose
Code Linting	flake8 on src/ and tests/	Ensures code quality standards
Unit Tests	Install deps, download data, train, pytest	Validates all functionality
Model Training	Full training + artifact verification	Ensures reproducibility





7.3 Pipeline Features

- Runs automatically on every push to main branch
- Jobs run sequentially — later jobs depend on earlier ones passing
- Artifacts saved for 7 days including model files and test results
- Pipeline fails immediately on any error with clear logs

8. Model Containerization

8.1 FastAPI Application

A FastAPI application was developed to serve the model predictions. The API exposes the following endpoints:

Endpoint	Method	Description
/	GET	Root health check
/health	GET	Detailed health status
/predict	POST	Heart disease prediction
/metrics	GET	Prometheus metrics
/docs	GET	Auto-generated API documentation

8.2 Sample Request/Response

Sample JSON input for /predict endpoint:

```
{"age": 52, "sex": 1, "cp": 0, "trestbps": 125, "chol": 212,
"fbs": 0, "restecg": 1, "thalach": 168, "exang": 0,
"oldpeak": 1.0, "slope": 2, "ca": 2, "thal": 3}
```

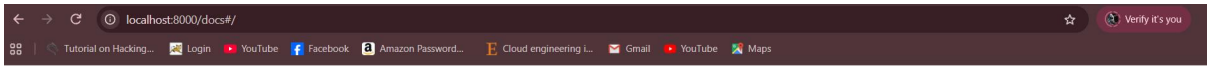
Sample JSON response:

```
{"prediction": 0, "prediction_label": "No Heart Disease",
"confidence": 0.8765, "probability_no_disease": 0.8765,
"probability_disease": 0.1235, "model_type": "RandomForestClassifier"}
```

8.3 Dockerfile

The Dockerfile uses python:3.11-slim as the base image, installs all dependencies from requirements.txt, copies the application code and model files, and runs the FastAPI server using Uvicorn on port 8000.

```
PS C:\Users\h\heart-disease-mlops> docker build -t heart-disease-api .
[+] Building 5.1s (15/15) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.1s
 => => transferring dockerfile: 1.27kB                                                                                              0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                                 2.8s
 => [internal] load .dockerignore                                                                                                   0.1s
 => => transferring context: 2B                                                                                                     0.0s
 => [ 1/10] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9ea65f6ce         0.2s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9ea65f6ce           0.1s
 => [internal] load build context                                                                                                   0.2s
 => => transferring context: 429.57kB                                                                                               0.1s
 => CACHED [ 2/10] WORKDIR /app                                                                                                     0.0s
 => CACHED [ 3/10] RUN apt-get update && apt-get install -y     gcc     g++     && rm -rf /var/lib/apt/lists/*                      0.0s
 => CACHED [ 4/10] COPY requirements.txt .                                                                                          0.0s
 => CACHED [ 5/10] RUN pip install --no-cache-dir --upgrade pip &&     pip install --no-cache-dir -r requirements.txt               0.0s
 => CACHED [ 6/10] COPY src/ ./src/                                                                                                 0.0s
 => CACHED [ 7/10] COPY api/ ./api/                                                                                                 0.0s
 => CACHED [ 8/10] COPY data/ ./data/                                                                                               0.0s
 => CACHED [ 9/10] COPY models/ ./models/                                                                                           0.0s
 => CACHED [10/10] RUN mkdir -p logs                                                                                                0.0s
 => exporting to image                                                                                                              0.6s
 => => exporting layers                                                                                                             0.0s
 => => exporting manifest sha256:5449b9e00c6321a8b7b386bc7e22f998b7a7abd710efde092c608b17b3f0c77c                                   0.0s
 => => exporting config sha256:20c833a9acc3296c5f27b561b8f61a3d8b098de1f30e78379f57b1859751a192                                     0.0s
 => => exporting attestation manifest sha256:bfcbfdbc2d50fdfb2e0ef1925efd2f277a78871377573c57d026ed2c5670c89e                       0.2s
 => => exporting manifest list sha256:b69b2df449e3b03b0475c296fc84f0c4cafb58c8d33d3fe68e05a8674fadd670                              0.1s
 => => naming to docker.io/library/heart-disease-api:latest                                                                         0.0s
 => => unpacking to docker.io/library/heart-disease-api:latest                                                                      0.0s
PS C:\Users\h\heart-disease-mlops> docker-compose up -d
WARN[0000] C:\Users\h\heart-disease-mlops\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential
confusion
[+] up 3/3
 ✓ Container grafana           Running                                                                                              0.0s
 ✓ Container heart-disease-api Running                                                                                              0.0s
 ✓ Container prometheus        Running                                                                                              0.0s
```



Heart Disease Prediction API

1.0.0 OAS 3.1

/openapi.json

AIMLCZG523 - MLOps Assignment 1

A cloud-ready API for predicting heart disease risk
based on patient health data.

Endpoints
- **GET /**   - Root health check
- **GET /health** - Detailed health status
- **POST /predict** - Predict heart disease risk
- **GET /metrics** - Prometheus metrics

default

GET /metrics Metrics

GET / Root

GET /health Health Check

POST /predict Predict

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "age": 52,
  "ca": 2,
  "chol": 212,
  "cp": 0,
  "exang": 0,
  "fbs": 0,
  "oldpeak": 1,
  "restecg": 1,
  "sex": 1,
  "slope": 2,
  "thal": 3,
  "thalach": 168,
  "trestbps": 125
}'
```

Request URL

```
http://localhost:8000/predict
```

Server response

Code	Details
200	Response body

```
{
  "prediction": 1,
  "prediction_label": "Heart Disease",
  "confidence": 0.8113,
  "probability_no_disease": 0.1887,
  "probability_disease": 0.8113,
  "model_type": "LogisticRegression",
  "timestamp": "2026-10-19T06:19:36.424272"
}
```

Download

9. Production Deployment

9.1 Kubernetes Setup

The Dockerized API was deployed to a local Kubernetes cluster using Minikube. The deployment uses 2 replicas for high availability and includes liveness and readiness probes for health monitoring.

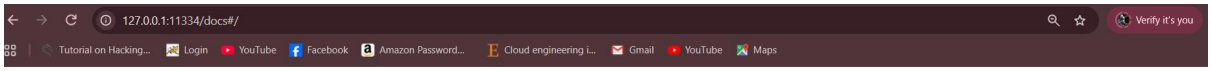
9.2 Deployment Configuration

Configuration	Value
Replicas	2
Container Port	8000
Service Type	LoadBalancer
Memory Request	256Mi
Memory Limit	512Mi
CPU Request	250m
CPU Limit	500m

9.3 Deployment Steps

- Started Minikube cluster with Docker driver
- Loaded Docker image into Minikube registry
- Applied deployment manifest (k8s/deployment.yaml)
- Applied service manifest (k8s/service.yaml)
- Exposed API via LoadBalancer service

```
PS C:\Users\h\heart-disease-mlops> kubectl get pods
NAME                                READY   STATUS    RESTARTS      AGE
heart-disease-api-6b9587ffdb-65gtp   1/1     Running   1 (10m ago)   13h
heart-disease-api-6b9587ffdb-m9t7z   1/1     Running   1 (10m ago)   13h
PS C:\Users\h\heart-disease-mlops>
PS C:\Users\h\heart-disease-mlops>
PS C:\Users\h\heart-disease-mlops> kubectl get services
NAME                        TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
heart-disease-api-service   LoadBalancer   10.105.48.75   <pending>     80:30080/TCP   13h
kubernetes                  ClusterIP      10.96.0.1      <none>        443/TCP        13h
PS C:\Users\h\heart-disease-mlops>
PS C:\Users\h\heart-disease-mlops>
PS C:\Users\h\heart-disease-mlops> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
heart-disease-api   2/2     2            2           13h
PS C:\Users\h\heart-disease-mlops>
PS C:\Users\h\heart-disease-mlops>
```



Heart Disease Prediction API

1.0.0 OAS 3.1

/openapi.json

AIMLCZG523 - MLOps Assignment 1

A cloud-ready API for predicting heart disease risk
based on patient health data.

Endpoints
- **GET /**   - Root health check
- **GET /health** - Detailed health status
- **POST /predict** - Predict heart disease risk
- **GET /metrics** - Prometheus metrics

default

GET `/metrics` Metrics

GET `/` Root

GET `/health` Health Check

POST `/predict` Predict

10. Monitoring and Logging

10.1 API Request Logging

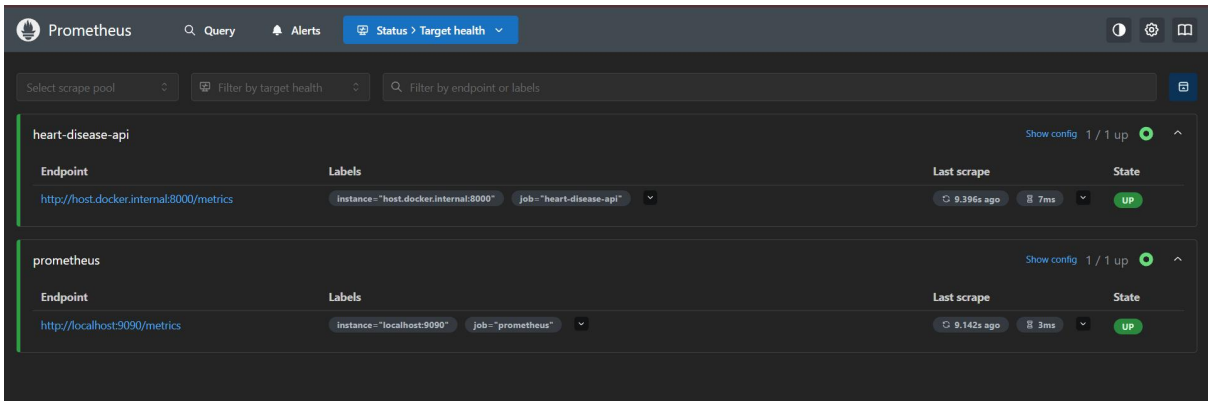
All API requests are logged using Python's logging module with the following details:

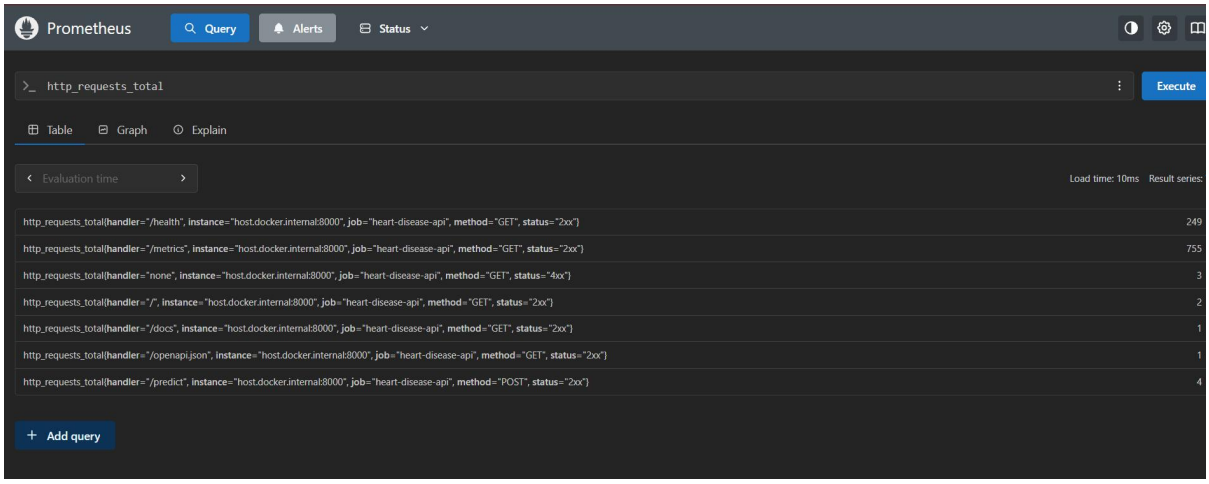
- Request method and path
- Response status code
- Request duration in seconds
- Prediction results and confidence scores
- Logs saved to api_requests.log file

10.2 Prometheus Metrics

Prometheus metrics are exposed at the /metrics endpoint using the prometheus-fastapi-instrumentator library. The following metrics are collected:

- http_requests_total — total number of HTTP requests
- http_request_duration_seconds — request duration histogram
- http_requests_in_progress — currently active requests

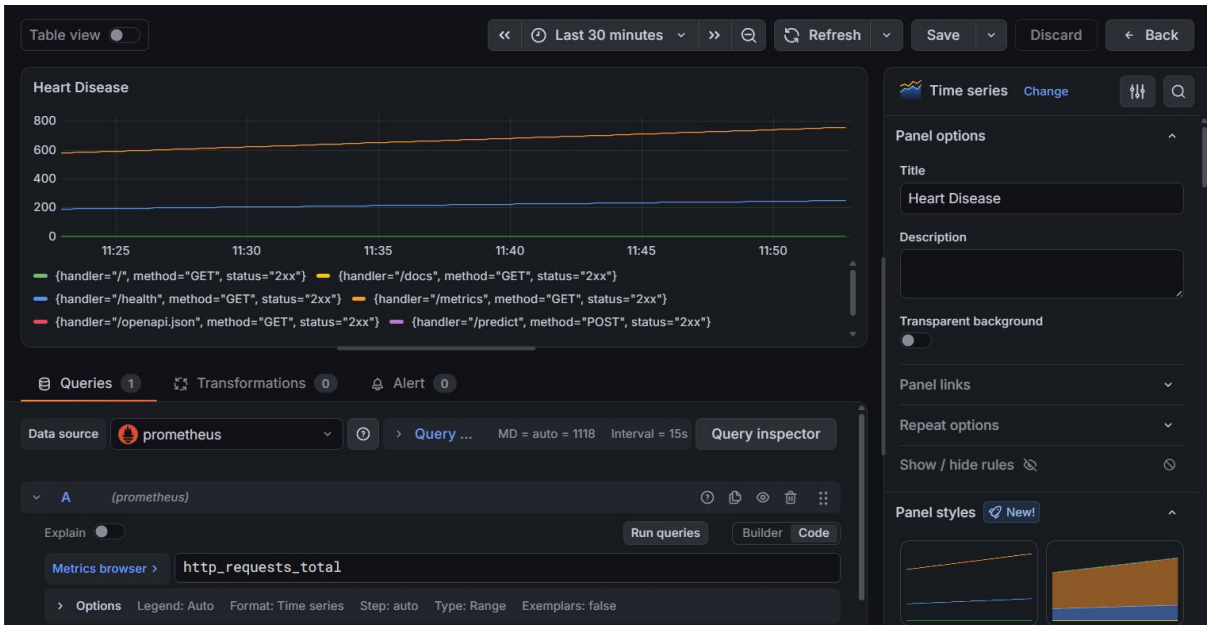




10.3 Grafana Dashboard

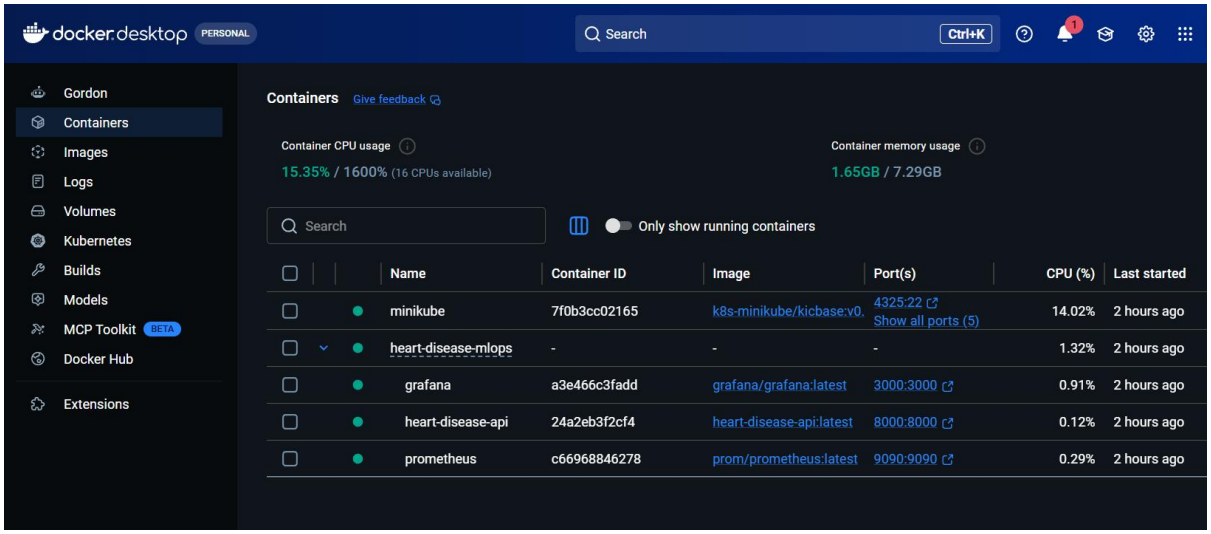
Grafana was configured with Prometheus as a data source. A dashboard was created to visualize API metrics including request rates, response times, and error rates.





10.4 Docker Compose Setup

All three services (API, Prometheus, Grafana) were orchestrated using Docker Compose for easy local deployment and testing.



11. Architecture Diagram

The following describes the complete MLOps architecture:

Layer	Component	Technology
Data	Dataset Storage	Local CSV / UCI Repository
Processing	EDA + Preprocessing	Python, Pandas, Scikit-learn
Training	Model Training	Scikit-learn (LR + RF)
Tracking	Experiment Tracking	MLflow + SQLite
Packaging	Model Serialization	Pickle + MLflow
Testing	Unit Testing	Pytest (41 tests)
CI/CD	Automated Pipeline	GitHub Actions
API	Model Serving	FastAPI + Uvicorn
Container	Containerization	Docker
Orchestration	Kubernetes Deployment	Minikube + kubectl
Monitoring	Metrics Collection	Prometheus
Visualization	Metrics Dashboard	Grafana

Architecture Flow:

Data Acquisition → EDA → Feature Engineering → Model Training → MLflow Tracking → Unit Tests → GitHub Actions CI/CD → Docker Container → Kubernetes Deployment → Prometheus + Grafana Monitoring

12. Conclusion

This assignment successfully demonstrated the design and implementation of a complete end-to-end MLOps pipeline for heart disease prediction. All 9 tasks were completed:

Task	Description	Status
1	Data Acquisition + EDA	Completed
2	Feature Engineering + Model Development	Completed
3	Experiment Tracking with MLflow	Completed
4	Model Packaging + Reproducibility	Completed
5	CI/CD Pipeline + Unit Tests	Completed
6	Model Containerization with Docker	Completed
7	Production Deployment on Kubernetes	Completed
8	Monitoring with Prometheus + Grafana	Completed
9	Documentation + Report	Completed

Key Achievements

- Built and deployed a classifier with ROC-AUC of ~0.89 for heart disease prediction
- Implemented complete MLflow experiment tracking with 3 runs logging parameters, metrics, artifacts and plots
- Created 41 unit tests with 100% pass rate
- Built automated CI/CD pipeline with GitHub Actions running linting, testing and training
- Containerized the API with Docker and deployed to Kubernetes with 2 replicas
- Set up production monitoring with Prometheus metrics and Grafana dashboard

Tools and Technologies

Category	Tool	Version
Language	Python	3.11
ML Framework	Scikit-learn	1.9.1
Experiment Tracking	MLflow	Latest
API Framework	FastAPI	Latest
Containerization	Docker	Latest
Orchestration	Minikube + kubectl	Latest
CI/CD	GitHub Actions	Latest
Monitoring	Prometheus + Grafana	Latest
Testing	Pytest	Latest
Data Processing	Pandas + NumPy	Latest

GitHub Repository

All code, notebooks, tests, Docker files, Kubernetes manifests, and CI/CD workflows are available at:

https://github.com/rpavansai1/heart-disease-mlops

Video Demo: https://drive.google.com/file/d/1aCCHL928LNw1HTDR3Arqy-7QtdkEYDgF/view?usp=sharing